

Flappy Bird AI with Q-Learning - Documentation

Overview

This project implements a reinforcement learning-based AI agent that learns to play the Flappy Bird game using **Q-Learning**. The game is implemented using **Pygame**, and the learning logic is based on **discretizing the state space** and using a Q-table to store and update action values.

Objectives

- Train an AI agent to play Flappy Bird using Q-learning.
 - Enable training from scratch or loading previous Q-tables.
 - Visualize training/play sessions.
 - Save Q-tables periodically.
-

Technologies Used

- Python 3.x
 - Pygame
 - NumPy
 - Pickle (for saving/loading Q-tables)
-

Q-Learning Basics

Key Components:

- **State Space:** Discretized distance to the next pipe (x) and vertical distance to the pipe gap (y).
- **Actions:**
 - 0: Do nothing

- 1: Jump
- **Q-table:** A 3D NumPy array `Q[x_bin][y_bin][action]`.

Learning Parameters:

- `alpha (learning_rate)` : Starts at 0.6, decays gradually.
 - `gamma (discount_factor)` : 0.95
 - `epsilon` : Starts at 1.0 (exploration), decays to 0.01 (exploitation).
-

Game Modes

1. Training Mode

- AI plays and updates the Q-table using rewards:
 - +1 for surviving
 - +10 * score for passing pipes
 - 100 for crashing
- Q-table is updated using the Q-learning update rule.

2. Play Mode

- Uses the trained Q-table (best Q-values) to play without learning.
 - Tracks best score for comparison.
-

Key Functions

`game_loop()`

Main game logic loop for training/play.

- Manages state transitions, rewards, Q-table updates.
- Handles rendering, collision detection, and pipe generation.

`ai_action(x, y)`

Chooses an action using epsilon-greedy policy.

`ai_best_action(x, y)`

Returns the best action from the Q-table (greedy).

`q_update(s, a, r, s')`

Updates the Q-value using the Q-learning formula:

$$Q(s, a) = Q(s, a) + \alpha * (\text{reward} + \gamma * \max(Q(s')) - Q(s, a))$$

`convert(x, y, pipes)`

Discretizes continuous bird and pipe positions into bins for indexing the Q-table.

`save_q_table()` and `load_q_table()`

Handles saving/loading Q-table to/from disk.

Interface Screens

Start Screen

- Options:
 - Train New AI
 - Watch AI Play
 - Load Q-Table

Load Screen

- Select from saved Q-tables by generation.
 - Use arrow keys and Enter.
-

Training Strategy

- Train for multiple generations.
 - Save Q-table every 10 generations.
 - Decay epsilon and learning rate after each generation.
 - Reward shaping to encourage pipe passing and penalize crashing.
-

File Structure

```
flappy_ai/  
|  
├── assets/      # Image files (bird, pipe, base, background)  
├── q_tables/    # Saved Q-table files  
├── main.py      # Main script containing game logic and training  
└── README.md    # Documentation
```

Future Improvements

- Use Deep Q-Network (DQN) for continuous state space.
- Add graphical analysis of training stats.
- Improve reward shaping strategy.

Author

- Developed by [Your Team Name/Your Name].

License

- MIT License